

GRADUATION PROJECT PRESENTATION - MODULE

Frontend Infrastructure, Shell & Localization

Refined visual systems, design tokens, next-intl translations, and dynamic RTL alignment.

Presenter: Member 2

Core Focus: Frontend Design & Localization

Menoufia University | Computer Science & Engineering Division

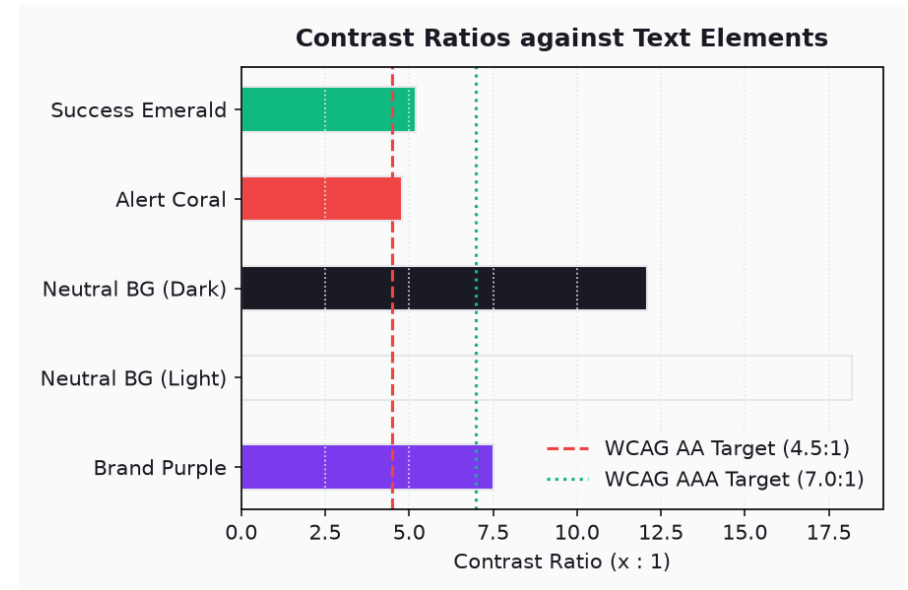
Frontend Layout & Next.js Framework

- React 19, Next.js 15, and TailwindCSS provide a fast, modular frontend architecture.
- Next.js layouts manage shared UI elements like the header, sidebar, and workspace containers.
- UI libraries like Radix UI ensure that dialogs, menus, and forms are fully accessible.
- [SCREENSHOT: VS Code project tree showing route structure directories under the app/ folder]

```
// Next.js App Router Structure
frontend/
|-- app/
|   |-- layout.tsx      (Shell Wrapper)
|   |-- page.tsx        (Landing View)
|   |-- notes/          (Notebook App)
|   |-- daily/          (Productivity App)
|-- components/         (Reusable widgets)
|-- hooks/              (State handlers)
```

Visual Tokens System & OKLCH Color Space

- We use the OKLCH color space for our styling tokens.
- Unlike HSL, OKLCH adjusts lightness to match human visual perception, preventing styling issues.
- Primary branding purple and neutral backgrounds transition smoothly between light and dark modes.
- [SCREENSHOT: Design system documentation showing color palettes, OKLCH values, and theme states]



Inter Typography Scale & Layout Grids

- Cortex uses a typographic scale based on the Inter font family for readability.
- Font sizes are structured to establish a clear reading hierarchy during long study sessions.
- Tailwind spacing tokens structure spacing grids across notes and planner views.
- [SCREENSHOT: Document view demonstrating typography scales, headers, and line-height spacing]

```
// Typography scale style tokens
h1 {
  font-size: 2rem;      /* 32px */
  font-weight: 700;
}
h2 {
  font-size: 1.5rem;    /* 24px */
  font-weight: 600;
}
body {
  font-size: 1rem;      /* 16px */
  line-height: 1.6;
}
```

Responsive App Shell Layout Grid

- AppShell sets up a flexible layout grid containing the app sidebar and main page.
- The sidebar collapses on smaller screens to maximize writing space.
- Dynamic hooks adjust styling parameters for touchscreen devices.
- [SCREENSHOT: Dashboard shell sidebar showing collapse buttons and navigation items]

```
// App Shell Structure Layout
export default function AppShell({ children }) {
  return (
    <div className="min-h-screen grid grid-cols-[auto_1fr]">
      <Sidebar className="h-full border-r" />
      <div className="flex flex-col overflow-hidden">
        <Header />
        <main className="flex-1 overflow-y-auto">
          {children}
        </main>
      </div>
    </div>
  );
}
```

Sidebar Mappings and Navigation Registry

- The navigation config maps sidebar lists to specific app page routes.
- Translation hooks label menu entries in the active language dynamically.
- Active state hooks highlight selected routes using color tokens.
- [SCREENSHOT: Sidebar navigation configuration file output and bindings]

```
// Navigation mappings registry config
export const NAV_ITEMS = [
  { label: "notes", path: "/notes", icon: "Notebook" },
  { label: "planner", path: "/daily", icon: "Calendar" },
  { label: "resources", path: "/data", icon: "Folder" },
  { label: "profile", path: "/profile", icon: "User" }
];
```

Bilingual Localization Key Configurations

- Internationalization (English/Arabic) is managed using next-intl middleware.
- Translation keys are organized inside JSON structures for easier updates.
- Localized assets load automatically based on the user's preferred language.
- [SCREENSHOT: Localized sidebar panels, comparing English labels with Arabic equivalent layout alignments]

```
// translation sample: en.json
{
  "common": {
    "search": "Search",
    "save": "Save"
  },
  "auth": {
    "signInTitle": "Welcome back",
    "signUpSubtitle": "Join Cortex and start building your second brain."
  }
}
```

Fluid RTL Layout Adaptations & Logical Styles

- The app shell dynamically checks the locale to toggle the HTML's direction (`dir='rtl'`).
- Tailwind CSS logical layout properties mirror margins, borders, and sidebar columns automatically.
- Arabic typography is adjusted to ensure clean spacing and reading comfort.
- [SCREENSHOT: Chrome DevTools CSS inspection showing logical margin alignments (ms-, me-)]

```
// Logical Spacing Mirrors
/* LTR Left Margin / RTL Right Margin */
.ms-4 {
  margin-inline-start: 1rem;
}
/* LTR Left Border / RTL Right Border */
.border-s {
  border-inline-start-width: 1px;
}
```


Accessibility Landmark & Accessibility Setup

- We verified color contrast ratios to meet WCAG AA requirements.
- WAI-ARIA landmark elements help screen readers navigate page containers.
- Keyboard focus outlines ensure that interactive elements are easy to navigate without a mouse.
- [SCREENSHOT: Lighthouse accessibility analysis scorecard, highlighting the contrast ratio rating]

```
// Accessible component with ARIA landmarks
export function AccessibleHeader() {
  return (
    <header role="banner" aria-label="Main Header">
      <button aria-label="Open navigation sidebar">
        <MenuIcon aria-hidden="true" />
      </button>
    </header>
  );
}
```

Next.js Server vs Client Rendering

- Cortex separates components into server components and client components.
- Server components load data directly from Supabase, reducing load times.
- Client components handle interactive elements, like rich-text editing and task updates.
- [SCREENSHOT: Network analyzer logs showing server rendering speeds and hydration benchmarks]

```
// Server Component Data Loading example
import { supabase } from "@/lib/supabase/server";

export default async function ServerPage() {
  const { data: recentNotes } = await supabase
    .from("notes")
    .select("id, title")
    .limit(5);

  return <ClientList notes={recentNotes} />;
}
```

Dynamic Theme Provider Configuration

- The theme provider manages light, dark, and system theme settings.
- We use CSS class bindings to load styling tokens dynamically.
- Static styles render page elements instantly, avoiding initial page flashes.
- [SCREENSHOT: Theme selector widget, showcasing light, dark, and system default choices]

```
// Theme providers setup wrapping
import { ThemeProvider } from "next-themes";

export function Providers({ children }) {
  return (
    <ThemeProvider
      attribute="class"
      defaultTheme="system"
      enableSystem
    >
      {children}
    </ThemeProvider>
  );
}
```